

2026-05-05-p1-f07-background-task-system

P1-F07 — Background Task System Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Add background-task execution to HelixCode: tools invoked with `run_in_background: true` run in goroutines and return a task ID immediately; new `TaskOutput` and `TaskStop` agent tools inspect/control them; `/tasks` slash command exposes the same to interactive users; shell tool streams line-oriented progress.

Architecture: Extend `internal/workflow/` with a new `background.go` file holding `BackgroundManager` + `BackgroundTask` (sibling to existing `Executor/Workflow`). Add a `BackgroundAware` opt-in interface in `internal/tools/`; only the shell tool implements it initially. The `ToolRegistry.Execute` dispatcher detects `run_in_background: true`, strips the flag, routes to `BackgroundManager.StartTask`. A sweeper goroutine reaps completed tasks older than 1h every 5 minutes.

Tech Stack: Go 1.26, testify v1.11, github.com/google/uuid v1.6 (already in go.mod), go.uber.org/zap (already in go.mod), github.com/spf13/cobra v1.8 (already in go.mod). **No new external dependencies.** Standard-library `os/exec`, `bufio`, `sync`, `sync/atomic`, `context`, `time`, `errors`.

Spec: `docs/superpowers/specs/2026-05-05-p1-f07-background-task-system-design.md` (commit d11885e)

Working directory for all go commands: `helix_code/` (the inner Go module). Git commands run from the meta-repo root `/run/media/milosvasic/DATA4TB/Projects/helix_code/` per the F01–F06 convention.

Anti-bluff smoke (run on every commit, FULL pattern):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|placeholder" internal/workflow/ internal/tools/task_tools.go internal/tools/shell/background.go internal/commands/tasks_command.go && echo "BLUFF FOUND" || echo "clean"
```

Task list

☐

P1-F07-T01 — bootstrap evidence + advance PROGRESS

☐

- ☐ P1-F07-T02 — workflow/background.go: BackgroundTask + TaskState (TDD)
 - ☐ P1-F07-T03 — workflow/background.go: BackgroundManager + sweeper (TDD)
 - ☐ P1-F07-T04 — tools/types_background.go: BackgroundAware interface + LineSink + error sentinel
 - ☐ P1-F07-T05 — tools/shell/background.go: shell BackgroundAware adapter (TDD)
 - ☐ P1-F07-T06 — tools/registry.go: SetBackgroundManager + Execute dispatch + adaptToolForBackground (TDD)
 - ☐ P1-F07-T07 — tools/task_tools.go: TaskOutputTool + TaskStopTool + RegisterTaskTools (TDD)
 - ☐ P1-F07-T08 — commands/tasks_command.go: /tasks slash command + builtin registration (TDD)
 - ☐ P1-F07-T09 — cmd/cli/main.go startup wiring + integration test (real subprocess)
 - ☐ P1-F07-T10 — Challenge with runtime evidence + cross-compile check
 - ☐ P1-F07-T11 — Feature 7 close-out + push to 4 remotes
-

Task 1: Bootstrap evidence + advance PROGRESS

Files: - Modify: docs/improvements/06_phase_1_evidence.md - Modify: docs/improvements/PROGRESS.md



Step 1: Append F07 section header to evidence file

Append to docs/improvements/06_phase_1_evidence.md:

P1-F07 — Background Task System (Ctrl+B)

****Spec:**** ``docs/superpowers/specs/2026-05-05-p1-f07-background-task-system-design.md` (commit `d11885e`)`

****Plan:**** ``docs/superpowers/plans/2026-05-05-p1-f07-background-task-system.md``

****Started:**** 2026-05-05

****Status:**** active

Task evidence trail

(filled in commit-by-commit as tasks land)



Step 2: Update PROGRESS.md current focus block

Replace the existing “## Current focus” block with:

Current focus

- ****Active phase:**** P1 – claude-code feature porting
- ****Active feature:**** F07 – Background Task System (Ctrl+B)
- ****Active task:**** P1-F07-T01 – bootstrap evidence + advance PROGRESS
- ****Last completed:**** P1-F06-T14 – Feature 6 (MCP Full Lifecycle) close-out + push
- ****Owner:**** agent (Claude Opus 4.7)
- ****Started:**** 2026-05-04
- ****Last touched:**** 2026-05-05
- ****Blocked-on:**** none



Step 3: Add F07 task list block to PROGRESS.md

After the existing F06 task list block (all 14 items checked), insert:

Active feature task list (P1-F07: Background Task System)

- [] P1-F07-T01 – bootstrap evidence + advance PROGRESS
- [] P1-F07-T02 – workflow/background.go: BackgroundTask + TaskState (TDD)
- [] P1-F07-T03 – workflow/background.go: BackgroundManager + sweeper (TDD)
- [] P1-F07-T04 – tools/types_background.go: BackgroundAware interface + LineSink + error sentinel
- [] P1-F07-T05 – tools/shell/background.go: shell BackgroundAware adapter (TDD)
- [] P1-F07-T06 – tools/registry.go: SetBackgroundManager + Execute dispatch + adaptToolForBackground (TDD)
- [] P1-F07-T07 – tools/task_tools.go: TaskOutputTool + TaskStopTool + RegisterTaskTools (TDD)
- [] P1-F07-T08 – commands/tasks_command.go: /tasks slash command + builtin registration (TDD)
- [] P1-F07-T09 – cmd/cli/main.go startup wiring + integration test (real subprocess)
- [] P1-F07-T10 – Challenge with runtime evidence + cross-compile check
- [] P1-F07-T11 – Feature 7 close-out + push



Step 4: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add docs/
    improvements/06_phase_1_evidence.md docs/improvements/
    PROGRESS.md
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
docs(P1-F07-T01): bootstrap Phase 1 / Feature 7 evidence +
    advance PROGRESS
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"

Task 2: workflow/background.go — BackgroundTask + TaskState (TDD)

Files: - Create: helix_code/internal/workflow/background.go (initial — types only; manager added in T03) - Create: helix_code/internal/workflow/background_test.go (initial)



Step 1: Write failing test

Create helix_code/internal/workflow/background_test.go:

```
package workflow

import (
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
)

func TestTaskState_String(t *testing.T) {
    assert.Equal(t, "pending", string(TaskPending))
    assert.Equal(t, "running", string(TaskRunning))
    assert.Equal(t, "completed", string(TaskCompleted))
    assert.Equal(t, "failed", string(TaskFailed))
    assert.Equal(t, "cancelled", string(TaskCancelled))
}

func TestBackgroundTask_AppendOutputAndLastLines(t *testing.T) {
    bt := newBackgroundTaskForTest("id-1", "Bash",
        map[string]any{"command": "x"}, 256, 4096)
    for i := 0; i < 10; i++ {
        bt.AppendOutput("line " + string(rune('0'+i)))
    }
}
```

```

    last := bt.LastLines(3)
    assert.Equal(t, []string{"line 7", "line 8", "line 9"}, last)
    assert.Equal(t, []string{"line 5", "line 6", "line 7", "line 8", "line 9"}, bt.LastLines(0)) // default 5
}

func TestBackgroundTask_OutputRingBoundedAtCap(t *testing.T) {
    bt := newBackgroundTaskForTest("id-2", "Bash", nil, 4,
        4096) // cap 4
    for i := 0; i < 10; i++ {
        bt.AppendOutput("line " + string(rune('0'+i)))
    }
    // Only last 4 retained
    assert.Equal(t, 4, len(bt.LastLines(100)))
    assert.Equal(t, []string{"line 6", "line 7", "line 8", "line 9"}, bt.LastLines(100))
}

func TestBackgroundTask_LineTruncatedAtMax(t *testing.T) {
    bt := newBackgroundTaskForTest("id-3", "Bash", nil, 256,
        16) // max 16 bytes per line
    bt.AppendOutput("0123456789ABCDEF_truncated_part")
    assert.Equal(t, []string{"0123456789ABCDEF"},
        bt.LastLines(1))
}

func TestBackgroundTask_StateAtomicGetSet(t *testing.T) {
    bt := newBackgroundTaskForTest("id-4", "Bash", nil, 256,
        4096)
    assert.Equal(t, TaskPending, bt.State())
    bt.SetState(TaskRunning)
    assert.Equal(t, TaskRunning, bt.State())
    bt.SetState(TaskCompleted)
    assert.Equal(t, TaskCompleted, bt.State())
    // EndedAt set on terminal state
    assert.NotNil(t, bt.EndedAt)
}

func TestBackgroundTask_SetStateRunningDoesNotSetEndedAt(t
    *testing.T) {
    bt := newBackgroundTaskForTest("id-5", "Bash", nil, 256,
        4096)
    bt.SetState(TaskRunning)
    assert.Nil(t, bt.EndedAt)
}

func TestBackgroundTask_ResultRoundTrip(t *testing.T) {

```

```

    bt := newBackgroundTaskForTest("id-6", "Bash", nil, 256,
        4096)
    bt.setResult("ok", nil)
    res, err := bt.Result()
    assert.Equal(t, "ok", res)
    assert.NoError(t, err)
}

// Test helper: newBackgroundTaskForTest is a package-internal
// constructor
// that bypasses BackgroundManager. It allows unit-testing
// BackgroundTask
// without spinning up the manager goroutine.
func newBackgroundTaskForTest(id, tool string, args
    map[string]any, cap, lineMax int) *BackgroundTask {
    return newBackgroundTask(id, tool, args, cap, lineMax, nil,
        nil)
}

```



Step 2: Run test to verify it fails

```

cd HelixCode && go test -count=1 -run "TestTaskState|
    TestBackgroundTask" ./internal/workflow/...

```

Expected: FAIL with undefined: TaskPending, BackgroundTask, newBackgroundTask, etc.



Step 3: Write workflow/background.go with types only

Create helix_code/internal/workflow/background.go:

```

// Package workflow's background.go provides BackgroundManager
// and BackgroundTask
// for running tools asynchronously in goroutines with line-
// oriented progress
// streaming and bounded output retention. This is the F07
// surface; it is a
// sibling to the existing multi-step Executor/Workflow types in
// the same
// package and does not depend on them.
package workflow

import (
    "context"
    "fmt"
    "sync"
    "sync/atomic"
    "time"

```

```

)

// TaskState is the lifecycle state of a background task.
type TaskState string

const (
    TaskPending    TaskState = "pending"
    TaskRunning    TaskState = "running"
    TaskCompleted TaskState = "completed"
    TaskFailed     TaskState = "failed"
    TaskCancelled  TaskState = "cancelled"
)

// IsTerminal reports whether the state is one in which the task
// is done.
func (s TaskState) IsTerminal() bool {
    return s == TaskCompleted || s == TaskFailed || s ==
        TaskCancelled
}

// BackgroundTask is one async tool execution.
type BackgroundTask struct {
    ID            string
    ToolName      string
    Args          map[string]any
    StartedAt     time.Time
    EndedAt       *time.Time

    state      atomic.Int32 // TaskState as int32
    mu         sync.RWMutex
    output     []string
    outputCap  int
    lineBytesMax int
    result     any
    err        error
    ctx        context.Context
    cancel     context.CancelFunc
}

// newBackgroundTask constructs a BackgroundTask. Used by the
// manager.
// ctx and cancel may be nil for unit tests that bypass the
// manager.
func newBackgroundTask(id, toolName string, args map[string]any,
    outputCap, lineBytesMax int,
    ctx context.Context, cancel context.CancelFunc)
    *BackgroundTask {
    if outputCap <= 0 {

```

```

        outputCap = 256
    }
    if lineBytesMax <= 0 {
        lineBytesMax = 4096
    }
    bt := &BackgroundTask{
        ID:            id,
        ToolName:      toolName,
        Args:          args,
        StartedAt:     time.Now(),
        outputCap:     outputCap,
        lineBytesMax:  lineBytesMax,
        ctx:           ctx,
        cancel:        cancel,
    }
    bt.state.Store(int32(taskStateOrdinal(TaskPending)))
    return bt
}

// State returns the current state via atomic load.
func (bt *BackgroundTask) State() TaskState {
    return ordinalToTaskState(bt.state.Load())
}

// SetState updates the state. On terminal transitions, EndedAt
// is set.
func (bt *BackgroundTask) SetState(s TaskState) {
    bt.state.Store(int32(taskStateOrdinal(s)))
    if s.IsTerminal() {
        bt.mu.Lock()
        now := time.Now()
        bt.EndedAt = &now
        bt.mu.Unlock()
    }
}

// AppendOutput adds a line to the bounded output ring. Lines
// longer than
// lineBytesMax are truncated. When the ring exceeds outputCap,
// the oldest
// line is dropped.
func (bt *BackgroundTask) AppendOutput(line string) {
    if len(line) > bt.lineBytesMax {
        line = line[:bt.lineBytesMax]
    }
    bt.mu.Lock()
    defer bt.mu.Unlock()
    bt.output = append(bt.output, line)
}

```



```

        if len(bt.output) > bt.outputCap {
            drop := len(bt.output) - bt.outputCap
            bt.output = append([]string(nil), bt.output[drop:]...)
        }
    }

// LastLines returns the last n lines (n<=0 means default 5).
func (bt *BackgroundTask) LastLines(n int) []string {
    if n <= 0 {
        n = 5
    }
    bt.mu.RLock()
    defer bt.mu.RUnlock()
    if len(bt.output) <= n {
        out := make([]string, len(bt.output))
        copy(out, bt.output)
        return out
    }
    out := make([]string, n)
    copy(out, bt.output[len(bt.output)-n:])
    return out
}

// setResult records the final tool result. Called by the manager
// goroutine.
func (bt *BackgroundTask) setResult(res any, err error) {
    bt.mu.Lock()
    bt.result = res
    bt.err = err
    bt.mu.Unlock()
}

// Result returns the final (result, err) tuple. Meaningful only
// after
// the task reaches a terminal state.
func (bt *BackgroundTask) Result() (any, error) {
    bt.mu.RLock()
    defer bt.mu.RUnlock()
    return bt.result, bt.err
}

// Err returns the recorded error (terminal-state convenience).
func (bt *BackgroundTask) Err() error {
    bt.mu.RLock()
    defer bt.mu.RUnlock()
    return bt.err
}

```

```
// taskStateOrdinal maps TaskState to a stable ordinal for
    atomic.Int32.
func taskStateOrdinal(s TaskState) int {
    switch s {
    case TaskPending:
        return 0
    case TaskRunning:
        return 1
    case TaskCompleted:
        return 2
    case TaskFailed:
        return 3
    case TaskCancelled:
        return 4
    default:
        return -1
    }
}

func ordinalToTaskState(o int32) TaskState {
    switch o {
    case 0:
        return TaskPending
    case 1:
        return TaskRunning
    case 2:
        return TaskCompleted
    case 3:
        return TaskFailed
    case 4:
        return TaskCancelled
    default:
        return TaskState(fmt.Sprintf("unknown(%d)", o))
    }
}
```



Step 4: Run tests to verify they pass

```
cd HelixCode && go test -count=1 -race -run "TestTaskState|
    TestBackgroundTask" ./internal/workflow/...
```

Expected: PASS (7/7 — TestTaskState_String,
 TestBackgroundTask_AppendOutputAndLastLines,
 TestBackgroundTask_OutputRingBoundedAtCap, TestBackgroundTask_LineTruncatedAtMax,
 TestBackgroundTask_StateAtomicGetSet,
 TestBackgroundTask_SetStateRunningDoesNotSetEndedAt,
 TestBackgroundTask_ResultRoundTrip).



Step 5: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" internal/workflow/ && echo "BLUFF FOUND" ||
echo "clean"
```

Expected: clean.



Step 6: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/workflow/background.go helix_code/
    internal/workflow/background_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F07-T02): add BackgroundTask + TaskState with bounded
output ring"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"

Task 3: workflow/background.go — BackgroundManager + sweeper (TDD)

Files: - Modify: helix_code/internal/workflow/background.go (append manager) - Modify: helix_code/internal/workflow/background_test.go (append tests)



Step 1: Write failing manager tests

Append to helix_code/internal/workflow/background_test.go:

```
import (
    "context"
    "errors"
    "sync"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    "go.uber.org/zap"
)
```

```

func newTestManager(t *testing.T, cfg ManagerConfig)
    *BackgroundManager {
    t.Helper()
    m := NewBackgroundManager(zap.NewNop(), cfg)
    t.Cleanup(func() { _ = m.Close() })
    return m
}

func TestBackgroundManager_StartTaskRunsExecutor(t *testing.T) {
    m := newTestManager(t, ManagerConfig{})
    exec := func(ctx context.Context, args map[string]any, sink
        LineSink) (any, error) {
        sink("a")
        sink("b")
        sink("c")
        return "result-x", nil
    }
    task, err := m.StartTask("FakeTool", map[string]any{"k":
        "v"}, exec)
    require.NoError(t, err)
    require.NotNil(t, task)
    require.NotEmpty(t, task.ID)
    // wait for completion
    require.Eventually(t, func() bool {
        return task.State() == TaskCompleted
    }, 2*time.Second, 10*time.Millisecond)
    assert.Equal(t, []string{"a", "b", "c"}, task.LastLines(10))
    res, terr := task.Result()
    assert.Equal(t, "result-x", res)
    assert.NoError(t, terr)
}

func TestBackgroundManager_StopTaskCancelsCtx(t *testing.T) {
    m := newTestManager(t, ManagerConfig{})
    exec := func(ctx context.Context, args map[string]any, sink
        LineSink) (any, error) {
        <-ctx.Done()
        return nil, ctx.Err()
    }
    task, err := m.StartTask("FakeTool", nil, exec)
    require.NoError(t, err)
    // wait until Running
    require.Eventually(t, func() bool {
        return task.State() == TaskRunning
    }, 1*time.Second, 10*time.Millisecond)
    require.NoError(t, m.StopTask(task.ID))
    require.Eventually(t, func() bool {
        return task.State() == TaskCancelled
    }, 1*time.Second, 10*time.Millisecond)
}

```

```

    }, 1*time.Second, 10*time.Millisecond)
}

func TestBackgroundManager_StopUnknownTaskReturnsError(t
    *testing.T) {
    m := newTestManager(t, ManagerConfig{})
    err := m.StopTask("nonexistent")
    assert.True(t, errors.Is(err, ErrTaskNotFound))
}

func TestBackgroundManager_StopAlreadyDoneTaskRejects(t
    *testing.T) {
    m := newTestManager(t, ManagerConfig{})
    exec := func(ctx context.Context, args map[string]any, sink
        LineSink) (any, error) { return "ok", nil }
    task, err := m.StartTask("FakeTool", nil, exec)
    require.NoError(t, err)
    require.Eventually(t, func() bool { return task.State() ==
        TaskCompleted }, 1*time.Second, 10*time.Millisecond)
    err = m.StopTask(task.ID)
    assert.True(t, errors.Is(err, ErrTaskNotRunning))
}

func TestBackgroundManager_GetTaskMissing(t *testing.T) {
    m := newTestManager(t, ManagerConfig{})
    _, err := m.GetTask("missing")
    assert.True(t, errors.Is(err, ErrTaskNotFound))
}

func TestBackgroundManager_PanickingExecRecovers(t *testing.T) {
    m := newTestManager(t, ManagerConfig{})
    exec := func(ctx context.Context, args map[string]any, sink
        LineSink) (any, error) {
        panic("boom")
    }
    task, err := m.StartTask("Panicker", nil, exec)
    require.NoError(t, err)
    require.Eventually(t, func() bool { return task.State() ==
        TaskFailed }, 1*time.Second, 10*time.Millisecond)
    _, terr := task.Result()
    require.Error(t, terr)
    assert.Contains(t, terr.Error(), "panic")
    // Manager survives, can start another task
    exec2 := func(ctx context.Context, args map[string]any, sink
        LineSink) (any, error) { return "ok", nil }
    task2, err := m.StartTask("Survivor", nil, exec2)
    require.NoError(t, err)

```

```

        require.Eventually(t, func() bool { return task2.State() ==
            TaskCompleted }, 1*time.Second, 10*time.Millisecond)
    }

```

```

func TestBackgroundManager_NilResultNilError(t *testing.T) {
    m := newTestManager(t, ManagerConfig{})
    exec := func(ctx context.Context, args map[string]any, sink
        LineSink) (any, error) {
        return nil, nil
    }
    task, err := m.StartTask("FakeTool", nil, exec)
    require.NoError(t, err)
    require.Eventually(t, func() bool { return task.State() ==
        TaskCompleted }, 1*time.Second, 10*time.Millisecond)
    res, terr := task.Result()
    assert.Nil(t, res)
    assert.NoError(t, terr)
}

```

```

func TestBackgroundManager_MaxConcurrentEnforced(t *testing.T) {
    m := newTestManager(t, ManagerConfig{MaxConcurrent: 2})
    block := make(chan struct{})
    exec := func(ctx context.Context, args map[string]any, sink
        LineSink) (any, error) {
        <-block
        return nil, nil
    }
    t1, err := m.StartTask("FakeTool", nil, exec)
    require.NoError(t, err)
    t2, err := m.StartTask("FakeTool", nil, exec)
    require.NoError(t, err)
    _, err = m.StartTask("FakeTool", nil, exec)
    assert.True(t, errors.Is(err, ErrTooManyTasks))
    close(block)
    _ = t1
    _ = t2
}

```

```

func TestBackgroundManager_SweeperReapsCompleted(t *testing.T) {
    m := newTestManager(t, ManagerConfig{
        SweepInterval: 50 * time.Millisecond,
        MaxAge:         100 * time.Millisecond,
    })
    exec := func(ctx context.Context, args map[string]any, sink
        LineSink) (any, error) { return "ok", nil }
    task, err := m.StartTask("FakeTool", nil, exec)
    require.NoError(t, err)
}

```

```

require.Eventually(t, func() bool { return task.State() ==
    TaskCompleted }, 1*time.Second, 10*time.Millisecond)
require.Eventually(t, func() bool {
    _, gerr := m.GetTask(task.ID)
    return errors.Is(gerr, ErrTaskNotFound)
}, 2*time.Second, 50*time.Millisecond)
}

func TestBackgroundManager_SweeperLeavesRunning(t *testing.T) {
    m := newTestManager(t, ManagerConfig{
        SweepInterval: 50 * time.Millisecond,
        MaxAge:         50 * time.Millisecond,
    })
    block := make(chan struct{})
    exec := func(ctx context.Context, args map[string]any, sink
        LineSink) (any, error) {
        <-block
        return nil, nil
    }
    task, err := m.StartTask("FakeTool", nil, exec)
    require.NoError(t, err)
    require.Eventually(t, func() bool { return task.State() ==
        TaskRunning }, 1*time.Second, 10*time.Millisecond)
    // Sleep across multiple sweep ticks; running task must still
    exist.
    time.Sleep(300 * time.Millisecond)
    got, err := m.GetTask(task.ID)
    require.NoError(t, err)
    assert.Equal(t, task, got)
    close(block)
}

func TestBackgroundManager_CloseIdempotent(t *testing.T) {
    m := NewBackgroundManager(zap.NewNop(), ManagerConfig{})
    require.NoError(t, m.Close())
    require.NoError(t, m.Close())
    exec := func(ctx context.Context, args map[string]any, sink
        LineSink) (any, error) { return nil, nil }
    _, err := m.StartTask("X", nil, exec)
    assert.True(t, errors.Is(err, ErrManagerClosed))
}

func TestBackgroundManager_CloseCancelsInFlight(t *testing.T) {
    m := NewBackgroundManager(zap.NewNop(), ManagerConfig{})
    exec := func(ctx context.Context, args map[string]any, sink
        LineSink) (any, error) {
        <-ctx.Done()
        return nil, ctx.Err()
    }

```

```

    }
    task1, err := m.StartTask("X", nil, exec)
    require.NoError(t, err)
    task2, err := m.StartTask("Y", nil, exec)
    require.NoError(t, err)
    require.Eventually(t, func() bool {
        return task1.State() == TaskRunning && task2.State() ==
            TaskRunning
    }, 1*time.Second, 10*time.Millisecond)
    require.NoError(t, m.Close())
    assert.True(t, task1.State() == TaskCancelled ||
        task1.State() == TaskFailed)
    assert.True(t, task2.State() == TaskCancelled ||
        task2.State() == TaskFailed)
}

func TestBackgroundManager_ListTasksSnapshot(t *testing.T) {
    m := newTestManager(t, ManagerConfig{})
    var wg sync.WaitGroup
    for i := 0; i < 3; i++ {
        wg.Add(1)
        exec := func(ctx context.Context, args map[string]any,
            sink LineSink) (any, error) {
            defer wg.Done()
            return "ok", nil
        }
        _, err := m.StartTask("FakeTool", nil, exec)
        require.NoError(t, err)
    }
    wg.Wait()
    list := m.ListTasks()
    assert.Len(t, list, 3)
}

```



Step 2: Run failing tests

```

cd HelixCode && go test -count=1 -run "TestBackgroundManager" ./
    internal/workflow/...

```

Expected: FAIL with undefined: NewBackgroundManager, ManagerConfig, LineSink, ErrTaskNotFound, etc.



Step 3: Append manager + sweeper to background.go

Append to helix_code/internal/workflow/background.go:

```

import (
    "errors"

```



```

    "github.com/google/uuid"
    "go.uber.org/zap"
)

// LineSink is invoked by an executor for each line of progress
// output.
type LineSink func(line string)

// BackgroundExecutor is the closure StartTask runs in a
// goroutine.
type BackgroundExecutor func(ctx context.Context, args
    map[string]any, sink LineSink) (any, error)

// ManagerConfig configures a BackgroundManager.
type ManagerConfig struct {
    OutputCap      int           // per-task ring; default 256
    LineBytesMax   int           // per-line cap; default 4096
    SweepInterval  time.Duration // sweeper tick; default 5min
    MaxAge         time.Duration // post-completion retention;
    default 1h
    MaxConcurrent  int           // concurrent in-flight limit;
    default 64
}

// BackgroundManager manages concurrent background tasks.
type BackgroundManager struct {
    tasks    map[string]*BackgroundTask
    mu       sync.RWMutex
    cfg      ManagerConfig
    log      *zap.Logger
    closeCh  chan struct{}
    closed   bool
    wg       sync.WaitGroup
}

// Error sentinels.
var (
    ErrTaskNotFound      =
        errors.New("workflow: background task not found")
    ErrTaskNotRunning    = errors.New("workflow: task is not
        running")
    ErrManagerClosed     = errors.New("workflow: background manager
        closed")
    ErrTooManyTasks      =
        errors.New("workflow: too many concurrent background
        tasks")
)

```

```

// NewBackgroundManager constructs a manager and starts the
// sweeper goroutine.
func NewBackgroundManager(log *zap.Logger, cfg ManagerConfig)
    *BackgroundManager {
    if log == nil {
        log = zap.NewNop()
    }
    if cfg.OutputCap <= 0 {
        cfg.OutputCap = 256
    }
    if cfg.LineBytesMax <= 0 {
        cfg.LineBytesMax = 4096
    }
    if cfg.SweepInterval <= 0 {
        cfg.SweepInterval = 5 * time.Minute
    }
    if cfg.MaxAge <= 0 {
        cfg.MaxAge = 1 * time.Hour
    }
    if cfg.MaxConcurrent <= 0 {
        cfg.MaxConcurrent = 64
    }
    bm := &BackgroundManager{
        tasks:    make(map[string]*BackgroundTask),
        cfg:        cfg,
        log:        log,
        closeCh:    make(chan struct{}),
    }
    bm.wg.Add(1)
    go bm.sweepLoop()
    return bm
}

// StartTask spawns a goroutine to run the executor. Returns
// immediately
// with the task; the goroutine writes terminal state on exit.
func (bm *BackgroundManager) StartTask(toolName string, args
    map[string]any, exec BackgroundExecutor)
    (*BackgroundTask, error) {
    bm.mu.Lock()
    if bm.closed {
        bm.mu.Unlock()
        return nil, ErrManagerClosed
    }
    if bm.countInFlightLocked() >= bm.cfg.MaxConcurrent {
        bm.mu.Unlock()
        return nil, ErrTooManyTasks
    }

```

```

    }
    id := uuid.New().String()
    ctx, cancel := context.WithCancel(context.Background())
    task := newBackgroundTask(id, toolName, args,
        bm.cfg.OutputCap, bm.cfg.LineBytesMax, ctx, cancel)
    bm.tasks[id] = task
    bm.mu.Unlock()

    bm.log.Info("background task started",
        zap.String("id", id), zap.String("tool", toolName))

    bm.wg.Add(1)
    go bm.run(task, exec)
    return task, nil
}

// run executes the task in a goroutine with panic recovery.
func (bm *BackgroundManager) run(task *BackgroundTask, exec
    BackgroundExecutor) {
    defer bm.wg.Done()
    task.SetState(TaskRunning)
    defer func() {
        if r := recover(); r != nil {
            err := fmt.Errorf("panic: %v", r)
            task.AppendOutput(err.Error())
            task.setResult(nil, err)
            task.SetState(TaskFailed)
            bm.log.Warn("background task panicked",
                zap.String("id", task.ID), zap.Any("panic", r))
        }
    }()
    res, err := exec(task.ctx, task.Args, task.AppendOutput)
    task.setResult(res, err)
    switch {
    case err != nil && errors.Is(err, context.Canceled):
        task.SetState(TaskCancelled)
        bm.log.Info("background task cancelled",
            zap.String("id", task.ID))
    case err != nil:
        task.AppendOutput(fmt.Sprintf("Error: %v", err))
        task.SetState(TaskFailed)
        bm.log.Warn("background task failed",
            zap.String("id", task.ID), zap.Error(err))
    default:
        task.SetState(TaskCompleted)
        bm.log.Info("background task completed",
            zap.String("id", task.ID))
    }
}

```

```

}

// GetTask returns a task by ID.
func (bm *BackgroundManager) GetTask(id string)
    (*BackgroundTask, error) {
    bm.mu.RLock()
    defer bm.mu.RUnlock()
    task, ok := bm.tasks[id]
    if !ok {
        return nil, fmt.Errorf("%w: %s", ErrTaskNotFound, id)
    }
    return task, nil
}

// StopTask cancels a running task. Returns ErrTaskNotRunning if
// the task
// is already in a terminal state.
func (bm *BackgroundManager) StopTask(id string) error {
    bm.mu.RLock()
    task, ok := bm.tasks[id]
    bm.mu.RUnlock()
    if !ok {
        return fmt.Errorf("%w: %s", ErrTaskNotFound, id)
    }
    st := task.State()
    if st != TaskRunning && st != TaskPending {
        return fmt.Errorf("%w: state=%s", ErrTaskNotRunning, st)
    }
    task.cancel()
    return nil
}

// ListTasks returns a snapshot of all current tasks.
func (bm *BackgroundManager) ListTasks() []*BackgroundTask {
    bm.mu.RLock()
    defer bm.mu.RUnlock()
    out := make([]*BackgroundTask, 0, len(bm.tasks))
    for _, t := range bm.tasks {
        out = append(out, t)
    }
    return out
}

// Status returns the current state and last output snapshot.
func (bm *BackgroundManager) Status(id string) (TaskState,
    []string, error) {
    task, err := bm.GetTask(id)
    if err != nil {

```

```

        return TaskState(""), nil, err
    }
    return task.State(), task.LastLines(0), nil
}

// Close stops the sweeper, cancels all in-flight tasks, and
// waits briefly
// for goroutines to exit. Idempotent.
func (bm *BackgroundManager) Close() error {
    bm.mu.Lock()
    if bm.closed {
        bm.mu.Unlock()
        return nil
    }
    bm.closed = true
    close(bm.closeCh)
    snap := make([]*BackgroundTask, 0, len(bm.tasks))
    for _, t := range bm.tasks {
        snap = append(snap, t)
    }
    bm.mu.Unlock()
    for _, t := range snap {
        if t.cancel != nil {
            t.cancel()
        }
    }
    done := make(chan struct{})
    go func() { bm.wg.Wait(); close(done) }()
    select {
    case <-done:
    case <-time.After(5 * time.Second):
        bm.log.Warn("background manager close: drain timeout (5s)")
    }
    return nil
}

// countInFlightLocked counts non-terminal tasks. Caller must
// hold bm.mu.
func (bm *BackgroundManager) countInFlightLocked() int {
    n := 0
    for _, t := range bm.tasks {
        if !t.State().IsTerminal() {
            n++
        }
    }
    return n
}

```

```
// sweepLoop runs in a goroutine and periodically prunes terminal
// tasks
// older than cfg.MaxAge.
func (bm *BackgroundManager) sweepLoop() {
    defer bm.wg.Done()
    ticker := time.NewTicker(bm.cfg.SweepInterval)
    defer ticker.Stop()
    for {
        select {
        case <-bm.closeCh:
            return
        case <-ticker.C:
            bm.sweep()
        }
    }
}

func (bm *BackgroundManager) sweep() {
    cutoff := time.Now().Add(-bm.cfg.MaxAge)
    bm.mu.Lock()
    defer bm.mu.Unlock()
    for id, t := range bm.tasks {
        if !t.State().IsTerminal() {
            continue
        }
        t.mu.RLock()
        ended := t.EndedAt
        t.mu.RUnlock()
        if ended != nil && ended.Before(cutoff) {
            delete(bm.tasks, id)
            bm.log.Debug("background task swept",
                zap.String("id", id))
        }
    }
}
```



Step 4: Verify imports merged correctly

The two import blocks (one in T02's initial file, one appended above) must be merged. Open `helix_code/internal/workflow/background.go` and ensure there is exactly ONE `import ( ... )` block at the top with these entries (sorted):

```
import (
    "context"
    "errors"
    "fmt"
    "sync"
```

```

"sync/atomic"
"time"

"github.com/google/uuid"
"go.uber.org/zap"
)

```

Then run `gofmt -w helix_code/internal/workflow/background.go`.



Step 5: Run tests

```
cd HelixCode && go test -count=1 -race ./internal/workflow/...
```

Expected: PASS — all T02 tests + all T03 tests (TestBackgroundManager_*).



Step 6: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" internal/workflow/ && echo "BLUFF FOUND" ||
echo "clean"
```

Expected: clean.



Step 7: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/workflow/background.go helix_code/
    internal/workflow/background_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F07-T03): add BackgroundManager with sweeper, panic
recovery, MaxConcurrent"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

```
EOF
)"
```

Task 4: tools/types_background.go — BackgroundAware interface

Files: - Create: `helix_code/internal/tools/types_background.go` - Create: `helix_code/internal/tools/types_background_test.go`



Step 1: Write failing test

Create `helix_code/internal/tools/types_background_test.go`:

```
package tools

import (
    "context"
    "testing"

    "github.com/stretchr/testify/assert"
)

// fakeBgTool implements BackgroundAware for testing.
type fakeBgTool struct {
    name          string
    streamLines []string
    finalResult any
}

func (f *fakeBgTool) Name() string {
    return f.name
}

func (f *fakeBgTool) Description() string {
    return "fake bg"
}

func (f *fakeBgTool) Schema() *ToolSchema {
    return &ToolSchema{Name: f.name}
}

func (f *fakeBgTool) Category() ToolCategory {
    return ToolCategoryGeneral
}

func (f *fakeBgTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error) {
    return f.finalResult, nil
}

func (f *fakeBgTool) ExecuteWithProgress(ctx context.Context,
    params map[string]interface{}, sink LineSink)
    (interface{}, error) {
    for _, l := range f.streamLines {
        sink(l)
    }
    return f.finalResult, nil
}

func TestBackgroundAware_InterfaceSatisfied(t *testing.T) {
    var _ BackgroundAware = (*fakeBgTool)(nil)
}

func TestLineSink_BasicCallback(t *testing.T) {
    var got []string
    var sink LineSink = func(line string) { got = append(got,
        line) }
    sink("a")
    sink("b")
}
```



```

    assert.Equal(t, []string{"a", "b"}, got)
}

func TestErrNoBackgroundMgr_IsExported(t *testing.T) {
    assert.NotNil(t, ErrNoBackgroundMgr)
    assert.Contains(t, ErrNoBackgroundMgr.Error(),
        "BackgroundManager")
}

```

Note: this test references existing types (Tool, ToolSchema, ToolCategory, ToolCategoryGeneral). Verify those names exist in `internal/tools/registry.go` before running. If the actual category constant differs (e.g., `CategoryGeneral`), use the actual name.



Step 2: Run failing test

```

cd HelixCode && go test -count=1 -run "TestBackgroundAware|
    TestLineSink|TestErrNoBackgroundMgr" ./internal/tools/

```

Expected: FAIL with undefined: BackgroundAware, LineSink, ErrNoBackgroundMgr.



Step 3: Write `types_background.go`

Create `helix_code/internal/tools/types_background.go`:

```

package tools

import (
    "context"
    "errors"
)

// LineSink is invoked by a BackgroundAware tool for each line of
// progress
// output. The sink is supplied by the BackgroundManager and
// routes lines
// into the BackgroundTask's bounded output ring.
type LineSink func(line string)

// BackgroundAware is implemented by tools that produce line-
// oriented
// progress output. Tools implementing this interface get
// streaming
// behavior under run_in_background:true. Tools that don't
// implement it
// fall back to "final result only" semantics.
type BackgroundAware interface {
    Tool

```

```

    ExecuteWithProgress(ctx context.Context, params
        map[string]interface{}, sink LineSink) (interface{},
        error)
}

// ErrNoBackgroundMgr is returned by ToolRegistry.Execute when
// params include
// run_in_background:true but no BackgroundManager has been wired
// via
// ToolRegistry.SetBackgroundManager.
var ErrNoBackgroundMgr = errors.New("tools: run_in_background
    requested but no BackgroundManager wired")

```



Step 4: Run tests to verify they pass

```

cd HelixCode && go test -count=1 -race -run "TestBackgroundAware|
    TestLineSink|TestErrNoBackgroundMgr" ./internal/tools/

```

Expected: PASS (3/3).



Step 5: Anti-bluff smoke

```

cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
    placeholder" internal/tools/types_background.go && echo
    "BLUFF FOUND" || echo "clean"

```

Expected: clean.



Step 6: Commit

```

git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/tools/types_background.go helix_code/
    internal/tools/types_background_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F07-T04): add BackgroundAware interface + LineSink +
    error sentinel

```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

EOF
)"

Task 5: tools/shell/background.go — shell BackgroundAware adapter (TDD)

Files: - Create: helix_code/internal/tools/shell/background.go - Create: helix_code/internal/tools/shell/background_test.go



Step 1: Read existing shell tool

```
cat helix_code/internal/tools/shell/shell.go
cat helix_code/internal/tools/shell/executor.go
```

Identify the existing tool struct name (likely ShellTool), the existing Execute signature, and the path through which subprocess is spawned. The new background.go file adds an ExecuteWithProgress method on the existing tool struct (or a wrapper) that uses pipes + bufio.Scanner.



Step 2: Write failing test

Create helix_code/internal/tools/shell/background_test.go:

```
package shell

import (
    "context"
    "runtime"
    "strings"
    "sync"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/tools"
)

func skipIfWindows(t *testing.T) {
    t.Helper()
    if runtime.GOOS == "windows" {
        t.Skip("SKIP-OK: shell test relies on POSIX bash; CI runs on Linux only for now")
    }
}

func TestShellBackgroundAware_StreamsLines(t *testing.T) {
    skipIfWindows(t)
    tool := newShellToolForTest()
```

```

require.Implements(t, (*tools.BackgroundAware)(nil), tool)

var mu sync.Mutex
var got []string
sink := tools.LineSink(func(line string) {
    mu.Lock()
    got = append(got, line)
    mu.Unlock()
})
ctx, cancel := context.WithTimeout(context.Background(),
    5*time.Second)
defer cancel()
_, err := tool.ExecuteWithProgress(ctx,
    map[string]interface{}{
        "command": "echo a; echo b; echo c",
    }, sink)
require.NoError(t, err)
mu.Lock()
defer mu.Unlock()
assert.Equal(t, []string{"a", "b", "c"}, got)
}

```

```

func TestShellBackgroundAware_ContextCancelKillsProcess(t
    *testing.T) {
    skipIfWindows(t)
    tool := newShellToolForTest()
    ctx, cancel := context.WithCancel(context.Background())
    done := make(chan error, 1)
    go func() {
        _, err := tool.ExecuteWithProgress(ctx,
            map[string]interface{}{
                "command": "sleep 30",
            }, func(string) {})
        done <- err
    }()
    time.Sleep(100 * time.Millisecond)
    cancel()
    select {
    case err := <-done:
        require.Error(t, err)
        // Either ctx error or signal-killed exit error is
        // acceptable.
    case <-time.After(3 * time.Second):
        t.Fatal("subprocess did not exit within 3s after ctx
            cancel")
    }
}

```

```

func TestShellBackgroundAware_ExitNonZeroIsErrorButCompletes(t
    *testing.T) {
    skipIfWindows(t)
    tool := newShellToolForTest()
    ctx, cancel := context.WithTimeout(context.Background(),
        3*time.Second)
    defer cancel()
    _, err := tool.ExecuteWithProgress(ctx,
        map[string]interface{}{
            "command": "exit 7",
        }, func(string) {})
    require.Error(t, err)
    assert.Contains(t, strings.ToLower(err.Error()), "exit")
}

```

`newShellToolForTest()` is a helper you write in `background_test.go` that constructs the existing shell tool with whatever sandbox/security defaults are appropriate. If the existing shell tool's constructor takes complex deps, define a minimal config that disables non-essential features for the test.



Step 3: Run failing test

```

cd HelixCode && go test -count=1 -run
    "TestShellBackgroundAware" ./internal/tools/shell/

```

Expected: FAIL with `ExecuteWithProgress` undefined or interface assertion failure.



Step 4: Write `background.go`

Create `helix_code/internal/tools/shell/background.go`:

```

package shell

import (
    "bufio"
    "context"
    "fmt"
    "io"
    "os/exec"
    "strings"
    "sync"

    "dev.helix.code/internal/tools"
)

// ExecuteWithProgress runs the shell command and streams stdout/
// stderr lines

```

```

// through sink. The final aggregated result is returned for
// compatibility with
// the existing shell tool's Execute return shape.
//
// This makes ShellTool implement tools.BackgroundAware. When a
// ToolRegistry
// dispatches with run_in_background:true, the BackgroundManager
// invokes this
// method instead of Execute, letting the BackgroundTask's output
// ring receive
// real-time progress.
func (t *ShellTool) ExecuteWithProgress(ctx context.Context,
    params map[string]interface{}, sink tools.LineSink)
    (interface{}, error) {
    command, ok := params["command"].(string)
    if !ok || command == "" {
        return nil, fmt.Errorf("shell: command must be a non-
            empty string")
    }

    // Use exec.CommandContext so ctx cancel kills the
    // subprocess.
    cmd := exec.CommandContext(ctx, "sh", "-c", command)
    if cwd, ok := params["cwd"].(string); ok && cwd != "" {
        cmd.Dir = cwd
    }

    stdout, err := cmd.StdoutPipe()
    if err != nil {
        return nil, fmt.Errorf("shell: stdout pipe: %w", err)
    }
    stderr, err := cmd.StderrPipe()
    if err != nil {
        return nil, fmt.Errorf("shell: stderr pipe: %w", err)
    }
    if err := cmd.Start(); err != nil {
        return nil, fmt.Errorf("shell: start: %w", err)
    }

    var (
        mu      sync.Mutex
        lines   []string
        appendLn = func(line string) {
            mu.Lock()
            lines = append(lines, line)
            mu.Unlock()
            sink(line)
        }
    )

```

```

)

scan := func(rd io.Reader, wg *sync.WaitGroup) {
    defer wg.Done()
    s := bufio.NewScanner(rd)
    s.Buffer(make([]byte, 4096), 1024*1024)
    for s.Scan() {
        appendLn(s.Text())
    }
}

var wg sync.WaitGroup
wg.Add(2)
go scan(stdout, &wg)
go scan(stderr, &wg)
wg.Wait()

waitErr := cmd.Wait()
mu.Lock()
output := strings.Join(lines, "\n")
mu.Unlock()
if waitErr != nil {
    return map[string]interface{}{
        "output":    output,
        "exit_code": exitCodeFromError(waitErr, cmd),
    }, fmt.Errorf("shell: command exit: %w", waitErr)
}
return map[string]interface{}{
    "output":    output,
    "exit_code": 0,
}, nil
}

// exitCodeFromError extracts the OS exit code from an
// exec.ExitError.
func exitCodeFromError(err error, cmd *exec.Cmd) int {
    if cmd != nil && cmd.ProcessState != nil {
        return cmd.ProcessState.ExitCode()
    }
    if ee, ok := err.(*exec.ExitError); ok {
        return ee.ExitCode()
    }
    return -1
}

```

If ShellTool is not the actual struct name in the existing shell.go, replace it with the actual name. The receiver type matters; the rest is independent.



Step 5: Run tests under -race

```
cd HelixCode && go test -count=1 -race -run
    "TestShellBackgroundAware" ./internal/tools/shell/
```

Expected: PASS (3/3).



Step 6: Confirm full shell package still passes

```
cd HelixCode && go test -count=1 -race ./internal/tools/shell/
```

Expected: PASS — shell's existing tests + the new BackgroundAware tests.



Step 7: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" internal/tools/shell/background.go && echo
"BLUFF FOUND" || echo "clean"
```

Expected: clean.



Step 8: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/tools/shell/background.go helix_code/
    internal/tools/shell/background_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F07-T05): shell tool implements BackgroundAware
(streaming stdout/stderr)

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"
```

Task 6: tools/registry.go — SetBackgroundManager + Execute dispatch (TDD)

Files: - Modify: helix_code/internal/tools/registry.go - Create: helix_code/internal/tools/registry_background_test.go



Step 1: Write failing test

Create helix_code/internal/tools/registry_background_test.go:


```

package tools

import (
    "context"
    "errors"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    "go.uber.org/zap"

    "dev.helix.code/internal/workflow"
)

// fakeStreamingTool implements BackgroundAware for end-to-end
// registry tests.
type fakeStreamingTool struct {
    name          string
    lines         []string
    finalResult   any
    gotParams     map[string]interface{}
}

func (f *fakeStreamingTool) Name() string
{ return f.name }
func (f *fakeStreamingTool) Description() string
{ return "" }
func (f *fakeStreamingTool) Schema() *ToolSchema
{ return &ToolSchema{Name: f.name} }
func (f *fakeStreamingTool) Category() ToolCategory
{ return ToolCategoryGeneral }
func (f *fakeStreamingTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error) {
    f.gotParams = params
    return f.finalResult, nil
}
func (f *fakeStreamingTool) ExecuteWithProgress(ctx
    context.Context, params map[string]interface{}, sink
    LineSink) (interface{}, error) {
    f.gotParams = params
    for _, l := range f.lines {
        sink(l)
    }
    return f.finalResult, nil
}

// fakePlainTool implements only Tool, not BackgroundAware.

```

```

type fakePlainTool struct {
    name          string
    finalResult   any
    gotParams     map[string]interface{}
}

func (f *fakePlainTool) Name() string {
    return f.name }
func (f *fakePlainTool) Description() string {
    return "" }
func (f *fakePlainTool) Schema() *ToolSchema {
    return &ToolSchema{Name: f.name} }
func (f *fakePlainTool) Category() ToolCategory {
    return ToolCategoryGeneral }
func (f *fakePlainTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error) {
    f.gotParams = params
    return f.finalResult, nil
}

func newRegistryWithBgMgr(t *testing.T) (*ToolRegistry,
    *workflow.BackgroundManager) {
    t.Helper()
    r, err := NewToolRegistry(DefaultRegistryConfig())
    require.NoError(t, err)
    bm := workflow.NewBackgroundManager(zap.NewNop(),
        workflow.ManagerConfig{})
    t.Cleanup(func() { _ = bm.Close() })
    r.SetBackgroundManager(bm)
    return r, bm
}

func TestRegistry_RunInBackgroundFlagDispatchesToManager(t
    *testing.T) {
    r, bm := newRegistryWithBgMgr(t)
    tool := &fakeStreamingTool{name: "Streamer", lines:
        []string{"x", "y"}, finalResult: "ok"}
    r.Register(tool)
    res, err := r.Execute(context.Background(), "Streamer",
        map[string]interface{}{
            "run_in_background": true,
            "key":                "value",
        })
    require.NoError(t, err)
    m, ok := res.(map[string]interface{})
    require.True(t, ok)
    taskID, ok := m["task_id"].(string)
    require.True(t, ok)
}

```

```

require.NotEmpty(t, taskID)

// Wait for completion
require.Eventually(t, func() bool {
    task, err := bm.GetTask(taskID)
    return err == nil && task.State() ==
        workflow.TaskCompleted
}, 2*time.Second, 10*time.Millisecond)

task, err := bm.GetTask(taskID)
require.NoError(t, err)
assert.Equal(t, []string{"x", "y"}, task.LastLines(10))
// Param flag stripped
assert.NotContains(t, tool.getParams, "run_in_background")
assert.Equal(t, "value", tool.getParams["key"])
}

func TestRegistry_RunInBackgroundWithoutManagerErrors(t
    *testing.T) {
    r, err := NewToolRegistry(DefaultRegistryConfig())
    require.NoError(t, err)
    tool := &fakeStreamingTool{name: "Streamer"}
    r.Register(tool)
    _, err = r.Execute(context.Background(), "Streamer",
        map[string]interface{}{
            "run_in_background": true,
        })
    require.Error(t, err)
    assert.True(t, errors.Is(err, ErrNoBackgroundMgr))
}

func TestRegistry_NonBackgroundAwareUsesFallback(t *testing.T) {
    r, bm := newRegistryWithBgMgr(t)
    tool := &fakePlainTool{name: "Plain", finalResult: "the-
        result"}
    r.Register(tool)
    res, err := r.Execute(context.Background(), "Plain",
        map[string]interface{}{
            "run_in_background": true,
        })
    require.NoError(t, err)
    m := res.(map[string]interface{})
    taskID := m["task_id"].(string)
    require.Eventually(t, func() bool {
        task, err := bm.GetTask(taskID)
        return err == nil && task.State() ==
            workflow.TaskCompleted
    }, 2*time.Second, 10*time.Millisecond)
}

```

```

task, _ := bm.GetTask(taskID)
// fallback path writes formatted final result into sink as a
// single line
last := task.LastLines(1)
require.Len(t, last, 1)
assert.Contains(t, last[0], "the-result")
}

func TestRegistry_BackgroundFlagFalseTakesForegroundPath(t
    *testing.T) {
    r, _ := newRegistryWithBgMgr(t)
    tool := &fakePlainTool{name: "Plain", finalResult: "fg"}
    r.Register(tool)
    res, err := r.Execute(context.Background(), "Plain",
        map[string]interface{}{
            "run_in_background": false,
        })
    require.NoError(t, err)
    // Foreground returns the tool's result directly, not a
    // task_id map.
    assert.Equal(t, "fg", res)
}

```



Step 2: Run failing test

```

cd HelixCode && go test -count=1 -run
    "TestRegistry_RunInBackground|
    TestRegistry_NonBackgroundAware|
    TestRegistry_BackgroundFlagFalse" ./internal/tools/

```

Expected: FAIL with `r.SetBackgroundManager` undefined.



Step 3: Modify registry.go

Open `helix_code/internal/tools/registry.go`. Add to the `ToolRegistry` struct:

```

type ToolRegistry struct {
    // ...existing fields...
    bgManager *workflow.BackgroundManager // F07: nil if not
    wired
}

```

Add the `dev.helix_code/internal/workflow` import.

Add the setter method (anywhere among the methods):

```

// SetBackgroundManager wires a BackgroundManager. Calls to
// Execute with
// run_in_background:true require this to be set. Optional; nil
// disables
// background dispatch (Execute returns ErrNoBackgroundMgr in
// that case).
func (r *ToolRegistry) SetBackgroundManager(m
    *workflow.BackgroundManager) {
    r.mu.Lock()
    defer r.mu.Unlock()
    r.bgManager = m
}

```

In Execute, near the very top (before fireBefore / tool resolution), add the dispatch check:

```

func (r *ToolRegistry) Execute(ctx context.Context, name string,
    params map[string]interface{}) (interface{}, error) {
    if bg, ok := params["run_in_background"].(bool); ok && bg {
        return r.executeInBackground(ctx, name, params)
    }
    // ...existing foreground logic stays unchanged below this
    // line...
}

```

Then add executeInBackground and adaptToolForBackground:

```

// executeInBackground routes the call to the BackgroundManager.
func (r *ToolRegistry) executeInBackground(ctx context.Context,
    name string, params map[string]interface{})
    (interface{}, error) {
    r.mu.RLock()
    bm := r.bgManager
    r.mu.RUnlock()
    if bm == nil {
        return nil, ErrNoBackgroundMgr
    }
    tool, err := r.Get(name)
    if err != nil {
        return nil, err
    }
    cleanArgs := stripBackgroundFlag(params)
    bgExec := r.adaptToolForBackground(tool)
    task, err := bm.StartTask(name, cleanArgs, bgExec)
    if err != nil {
        return nil, err
    }
    return map[string]interface{}{
        "task_id": task.ID,
        "state":   string(task.State()),
    }, nil
}

```

```

        "message": fmt.Sprintf("Task started in background. ID:
        %s — use TaskOutput to check progress.", task.ID),
    }, nil
}

// adaptToolForBackground returns a workflow.BackgroundExecutor
// that bridges
// the tool's Execute / ExecuteWithProgress methods. Streaming-
// aware tools
// get the sink directly; plain tools get a final-result-only
// fallback that
// writes the formatted result as a single sink line at
// completion.
func (r *ToolRegistry) adaptToolForBackground(tool Tool)
    workflow.BackgroundExecutor {
    if ba, ok := tool.(BackgroundAware); ok {
        return func(ctx context.Context, args
            map[string]interface{}, sink workflow.LineSink)
            (interface{}, error) {
                return ba.ExecuteWithProgress(ctx, args,
                    LineSink(sink))
            }
    }
    return func(ctx context.Context, args
        map[string]interface{}, sink workflow.LineSink)
        (interface{}, error) {
            res, err := tool.Execute(ctx, args)
            if err == nil && res != nil {
                sink(fmt.Sprintf("%v", res))
            }
            return res, err
        }
    }

func stripBackgroundFlag(params map[string]interface{})
    map[string]interface{} {
    out := make(map[string]interface{}, len(params))
    for k, v := range params {
        if k == "run_in_background" {
            continue
        }
        out[k] = v
    }
    return out
}

```

If `r.mu` doesn't exist or has a different name, use the actual field. If `Get` returns an error type other than what's shown, adapt. The two `LineSink` types — `tools.LineSink` (T04) and

workflow.LineSink (T03) — have the same shape; the conversion LineSink(sink) reinterprets the function value.



Step 4: Verify imports

helix_code/internal/tools/registry.go should now import dev.helix.code/internal/workflow.Run gofmt -w helix_code/internal/tools/registry.go.



Step 5: Run failing tests now passing

```
cd HelixCode && go test -count=1 -race -run
    "TestRegistry_RunInBackground|
    TestRegistry_NonBackgroundAware|
    TestRegistry_BackgroundFlagFalse" ./internal/tools/
```

Expected: PASS (4/4).



Step 6: Run full tools sweep

```
cd HelixCode && go test -count=1 -race ./internal/tools/
```

Expected: PASS — pre-existing tests + new T04+T06 tests. (Pre-existing failures in tools/git, tools/multiedit, tools/shell may persist; document if so but do not introduce new failures in internal/tools/ itself.)



Step 7: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" internal/tools/registry.go internal/tools/
types_background.go && echo "BLUFF FOUND" || echo "clean"
```

Expected: clean.



Step 8: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/tools/registry.go helix_code/
    internal/tools/registry_background_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F07-T06): ToolRegistry dispatches run_in_background flag
to BackgroundManager
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF
)"

Task 7: tools/task_tools.go — TaskOutputTool + TaskStopTool (TDD)

Files: - Create: helix_code/internal/tools/task_tools.go - Create: helix_code/internal/tools/task_tools_test.go



Step 1: Write failing test

Create helix_code/internal/tools/task_tools_test.go:

```
package tools

import (
    "context"
    "errors"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    "go.uber.org/zap"

    "dev.helix.code/internal/workflow"
)

func newTaskToolsManager(t *testing.T)
    *workflow.BackgroundManager {
    t.Helper()
    bm := workflow.NewBackgroundManager(zap.NewNop(),
        workflow.ManagerConfig{})
    t.Cleanup(func() { _ = bm.Close() })
    return bm
}

func startSimpleTask(t *testing.T, bm
    *workflow.BackgroundManager, lines []string) string {
    t.Helper()
    exec := func(ctx context.Context, args
        map[string]interface{}, sink workflow.LineSink)
        (interface{}, error) {
        for _, l := range lines {
            sink(l)
        }
        return "ok", nil
    }
```



```

    }
    task, err := bm.StartTask("FakeTool", nil, exec)
    require.NoError(t, err)
    require.Eventually(t, func() bool { return task.State() ==
        workflow.TaskCompleted }, 2*time.Second,
        10*time.Millisecond)
    return task.ID
}

```

```

func TestTaskOutputTool_ReturnsLastNLines(t *testing.T) {
    bm := newTaskToolsManager(t)
    id := startSimpleTask(t, bm, []string{"l1", "l2", "l3",
        "l4", "l5", "l6"})
    tool := NewTaskOutputTool(bm)
    res, err := tool.Execute(context.Background(),
        map[string]interface{}{
            "task_id": id,
            "lines": 3,
        })
    require.NoError(t, err)
    m := res.(map[string]interface{})
    assert.Equal(t, "l4\nl5\nl6", m["output"])
    assert.Equal(t, 3, m["line_count"])
}

```

```

func TestTaskOutputTool_DefaultLinesIs5(t *testing.T) {
    bm := newTaskToolsManager(t)
    id := startSimpleTask(t, bm, []string{"a", "b", "c", "d",
        "e", "f", "g"})
    tool := NewTaskOutputTool(bm)
    res, err := tool.Execute(context.Background(),
        map[string]interface{}{
            "task_id": id,
        })
    require.NoError(t, err)
    m := res.(map[string]interface{})
    assert.Equal(t, 5, m["line_count"])
}

```

```

func TestTaskOutputTool_UnknownTaskID(t *testing.T) {
    bm := newTaskToolsManager(t)
    tool := NewTaskOutputTool(bm)
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{
            "task_id": "missing",
        })
    require.Error(t, err)
    assert.True(t, errors.Is(err, workflow.ErrTaskNotFound))
}

```

```

}

func TestTaskStopTool_CancelsRunning(t *testing.T) {
    bm := newTaskToolsManager(t)
    exec := func(ctx context.Context, args
        map[string]interface{}, sink workflow.LineSink)
        (interface{}, error) {
            <-ctx.Done()
            return nil, ctx.Err()
        }
    task, err := bm.StartTask("Blocker", nil, exec)
    require.NoError(t, err)
    require.Eventually(t, func() bool { return task.State() ==
        workflow.TaskRunning }, 1*time.Second,
        10*time.Millisecond)

    tool := NewTaskStopTool(bm)
    res, err := tool.Execute(context.Background(),
        map[string]interface{}{"task_id": task.ID})
    require.NoError(t, err)
    m := res.(map[string]interface{})
    assert.Equal(t, "stopped", m["status"])

    require.Eventually(t, func() bool { return task.State() ==
        workflow.TaskCancelled }, 1*time.Second,
        10*time.Millisecond)
}

func TestTaskStopTool_NotRunningRejects(t *testing.T) {
    bm := newTaskToolsManager(t)
    id := startSimpleTask(t, bm, []string{"x"})
    tool := NewTaskStopTool(bm)
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{"task_id": id})
    require.Error(t, err)
    assert.True(t, errors.Is(err, workflow.ErrTaskNotRunning))
}

```



Step 2: Run failing test

```

cd HelixCode && go test -count=1 -run "TestTaskOutputTool|
    TestTaskStopTool" ./internal/tools/

```

Expected: FAIL with undefined: NewTaskOutputTool, NewTaskStopTool.



Step 3: Write task_tools.go

Create `helix_code/internal/tools/task_tools.go`:

```
package tools

import (
    "context"
    "fmt"
    "strings"

    "dev.helix.code/internal/workflow"
)

// TaskOutputTool reads the tail output of a background task.
type TaskOutputTool struct {
    manager *workflow.BackgroundManager
}

// NewTaskOutputTool returns the agent-callable TaskOutput tool.
func NewTaskOutputTool(m *workflow.BackgroundManager)
    *TaskOutputTool {
    return &TaskOutputTool{manager: m}
}

func (t *TaskOutputTool) Name() string { return
    "TaskOutput" }
func (t *TaskOutputTool) Description() string {
    return "Read the output of a background task. Returns the
        last N lines (default 5) plus the task's current state."
}
func (t *TaskOutputTool) Category() ToolCategory { return
    ToolCategoryGeneral }
func (t *TaskOutputTool) Schema() *ToolSchema {
    return &ToolSchema{
        Name: "TaskOutput",
        Description: t.Description(),
        Parameters: map[string]interface{}{
            "type": "object",
            "properties": map[string]interface{}{
                "task_id": map[string]interface{}{
                    "type": "string",
                    "description": "ID of the background task",
                },
                "lines": map[string]interface{}{
                    "type": "integer",
                    "description": "Number of trailing lines to
return (default 5)",
                },
            },
        },
        "required": []string{"task_id"},
    },
}
```

```

    },
}
}

func (t *TaskOutputTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error) {
    id, ok := params["task_id"].(string)
    if !ok || id == "" {
        return nil, fmt.Errorf("TaskOutput: task_id must be a
            non-empty string")
    }
    n := 5
    switch v := params["lines"].(type) {
    case float64:
        n = int(v)
    case int:
        n = v
    }
    state, lines, err := t.manager.Status(id)
    if err != nil {
        return nil, err
    }
    if n <= 0 {
        n = 5
    }
    if len(lines) > n {
        lines = lines[len(lines)-n:]
    }
    task, _ := t.manager.GetTask(id)
    totalLines := 0
    if task != nil {
        totalLines = len(task.LastLines(1<<30))
    }
    return map[string]interface{}{
        "task_id":      id,
        "state":        string(state),
        "output":       strings.Join(lines, "\n"),
        "line_count":   len(lines),
        "total_lines":  totalLines,
    }, nil
}

// TaskStopTool cancels a running background task.
type TaskStopTool struct {
    manager *workflow.BackgroundManager
}

// NewTaskStopTool returns the agent-callable TaskStop tool.

```

```

func NewTaskStopTool(m *workflow.BackgroundManager)
    *TaskStopTool {
    return &TaskStopTool{manager: m}
}

func (t *TaskStopTool) Name() string { return
    "TaskStop" }
func (t *TaskStopTool) Description() string { return
    "Cancel a running background task by ID." }
func (t *TaskStopTool) Category() ToolCategory { return
    ToolCategoryGeneral }
func (t *TaskStopTool) Schema() *ToolSchema {
    return &ToolSchema{
        Name: "TaskStop",
        Description: t.Description(),
        Parameters: map[string]interface{}{
            "type": "object",
            "properties": map[string]interface{}{
                "task_id": map[string]interface{}{
                    "type": "string",
                    "description": "ID of the task to cancel",
                },
            },
            "required": []string{"task_id"},
        },
    }
}

func (t *TaskStopTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error) {
    id, ok := params["task_id"].(string)
    if !ok || id == "" {
        return nil, fmt.Errorf("TaskStop: task_id must be a non-
            empty string")
    }
    if err := t.manager.StopTask(id); err != nil {
        return nil, err
    }
    return map[string]interface{}{
        "task_id": id,
        "status": "stopped",
    }, nil
}

// RegisterTaskTools registers TaskOutput and TaskStop in the
// registry,
// bound to the supplied BackgroundManager.

```

```
func (r *ToolRegistry) RegisterTaskTools(m
    *workflow.BackgroundManager) {
    r.Register(NewTaskOutputTool(m))
    r.Register(NewTaskStopTool(m))
}
```

The exact ToolSchema field names (Parameters, Description) must match the existing struct — read internal/tools/registry.go to confirm. Adjust if needed.



Step 4: Run tests

```
cd HelixCode && go test -count=1 -race -run "TestTaskOutputTool|
    TestTaskStopTool" ./internal/tools/
```

Expected: PASS (5/5).



Step 5: Anti-bluff smoke

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
    placeholder" internal/tools/task_tools.go && echo "BLUFF
    FOUND" || echo "clean"
```

Expected: clean.



Step 6: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/tools/task_tools.go helix_code/
    internal/tools/task_tools_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F07-T07): add TaskOutput + TaskStop agent tools and
    registration"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

```
EOF
)"
```

Task 8: commands/tasks_command.go — /tasks slash + builtin registration (TDD)

Files: - Create: helix_code/internal/commands/tasks_command.go - Create: helix_code/internal/commands/tasks_command_test.go - Modify: helix_code/internal/commands/builtin/register.go — add

RegisterBuiltinCommandsWithTasks - Create: helix_code/internal/commands/builtin/tasks_register_test.go



Step 1: Read existing command interface

```
grep -n "type CommandContext\|type CommandResult\|type Command
      interface\|func .* Execute" helix_code/internal/commands/
      types.go helix_code/internal/commands/registry.go 2>/dev/
      null | head -30
cat helix_code/internal/commands/builtin/hooks_register_test.go
2>/dev/null
```

Use the same Command interface pattern + RegisterBuiltinCommandsWithHooks style proven in F05 and F06.



Step 2: Write failing slash test

Create helix_code/internal/commands/tasks_command_test.go:

```
package commands

import (
    "context"
    "strings"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    "go.uber.org/zap"

    "dev.helix.code/internal/workflow"
)

func newTasksCommandWithMgr(t *testing.T) (*TasksCommand,
    *workflow.BackgroundManager) {
    t.Helper()
    bm := workflow.NewBackgroundManager(zap.NewNop(),
        workflow.ManagerConfig{})
    t.Cleanup(func() { _ = bm.Close() })
    return NewTasksCommand(bm), bm
}

func TestSlashTasks_ListEmpty(t *testing.T) {
    c, _ := newTasksCommandWithMgr(t)
    res, err := c.Execute(context.Background(),
        &CommandContext{Args: nil})
    require.NoError(t, err)
```

```

    assert.Contains(t, res.Output, "ID")
    assert.Contains(t, res.Output, "STATE")
}

func TestSlashTasks_ListShowsTask(t *testing.T) {
    c, bm := newTasksCommandWithMgr(t)
    exec := func(ctx context.Context, args
        map[string]interface{}, sink workflow.LineSink)
        (interface{}, error) {
            return "ok", nil
        }
    task, err := bm.StartTask("FakeTool", nil, exec)
    require.NoError(t, err)
    require.Eventually(t, func() bool { return task.State() ==
        workflow.TaskCompleted }, 1*time.Second,
        10*time.Millisecond)

    res, err := c.Execute(context.Background(),
        &CommandContext{Args: []string{"list"}})
    require.NoError(t, err)
    assert.Contains(t, res.Output, task.ID)
    assert.Contains(t, res.Output, "completed")
}

func TestSlashTasks_OutputReturnsLines(t *testing.T) {
    c, bm := newTasksCommandWithMgr(t)
    exec := func(ctx context.Context, args
        map[string]interface{}, sink workflow.LineSink)
        (interface{}, error) {
            sink("first")
            sink("second")
            sink("third")
            return "ok", nil
        }
    task, err := bm.StartTask("FakeTool", nil, exec)
    require.NoError(t, err)
    require.Eventually(t, func() bool { return task.State() ==
        workflow.TaskCompleted }, 1*time.Second,
        10*time.Millisecond)

    res, err := c.Execute(context.Background(),
        &CommandContext{Args: []string{"output", task.ID}})
    require.NoError(t, err)
    for _, want := range []string{"first", "second", "third"} {
        assert.True(t, strings.Contains(res.Output, want), "want
            %q in output", want)
    }
}

```



```

func TestSlashTasks_StopCancels(t *testing.T) {
    c, bm := newTasksCommandWithMgr(t)
    exec := func(ctx context.Context, args
        map[string]interface{}, sink workflow.LineSink)
        (interface{}, error) {
            <-ctx.Done()
            return nil, ctx.Err()
        }
    task, err := bm.StartTask("Blocker", nil, exec)
    require.NoError(t, err)
    require.Eventually(t, func() bool { return task.State() ==
        workflow.TaskRunning }, 1*time.Second,
        10*time.Millisecond)

    res, err := c.Execute(context.Background(),
        &CommandContext{Args: []string{"stop", task.ID}})
    require.NoError(t, err)
    assert.Contains(t, res.Output, "stopped")
    require.Eventually(t, func() bool { return task.State() ==
        workflow.TaskCancelled }, 1*time.Second,
        10*time.Millisecond)
}

func TestSlashTasks_UnknownSubcommand(t *testing.T) {
    c, _ := newTasksCommandWithMgr(t)
    _, err := c.Execute(context.Background(),
        &CommandContext{Args: []string{"bogus"}})
    require.Error(t, err)
}

```

If the actual CommandContext/CommandResult shape differs, adapt this test. Read `internal/commands/types.go` first.



Step 3: Run failing test

```

cd HelixCode && go test -count=1 -run "TestSlashTasks" ./
    internal/commands/

```

Expected: FAIL with NewTasksCommand undefined.



Step 4: Write tasks_command.go

Create `helix_code/internal/commands/tasks_command.go`:

```

package commands

```

```

import (

```

```

    "context"
    "fmt"
    "strings"
    "text/tabwriter"

    "dev.helix.code/internal/workflow"
)

// TasksCommand implements the /tasks slash command.
type TasksCommand struct {
    manager *workflow.BackgroundManager
}

// NewTasksCommand returns a /tasks command bound to a
// BackgroundManager.
func NewTasksCommand(m *workflow.BackgroundManager)
    *TasksCommand {
    return &TasksCommand{manager: m}
}

func (c *TasksCommand) Name() string { return "tasks" }
func (c *TasksCommand) Description() string { return
    "Inspect and control background tasks" }

// Execute runs the slash command. Subcommands: list (default),
// output <id>, stop <id>.
func (c *TasksCommand) Execute(ctx context.Context, cc
    *CommandContext) (*CommandResult, error) {
    if c.manager == nil {
        return nil, fmt.Errorf("tasks: manager not initialised")
    }
    args := cc.Args
    sub := "list"
    if len(args) > 0 {
        sub = args[0]
    }
    switch sub {
    case "list":
        return &CommandResult{Output: c.list()}, nil
    case "output":
        if len(args) < 2 {
            return nil, fmt.Errorf("/tasks output <id>")
        }
        return c.output(args[1])
    case "stop":
        if len(args) < 2 {
            return nil, fmt.Errorf("/tasks stop <id>")
        }
    }
}

```

```

        return c.stop(args[1])
    default:
        return nil, fmt.Errorf("/tasks: unknown subcommand %q
            (want list|output|stop)", sub)
    }
}

func (c *TasksCommand) list() string {
    var sb strings.Builder
    tw := tabwriter.NewWriter(&sb, 0, 0, 2, ' ', 0)
    fmt.Fprintln(tw, "ID\tTOOL\tSTATE\tSTARTED")
    for _, t := range c.manager.ListTasks() {
        fmt.Fprintf(tw, "%s\t%s\t%s\t%s\n",
            t.ID, t.ToolName, t.State(),
            t.StartedAt.Format("15:04:05"))
    }
    tw.Flush()
    return sb.String()
}

func (c *TasksCommand) output(id string) (*CommandResult, error)
{
    state, lines, err := c.manager.Status(id)
    if err != nil {
        return nil, err
    }
    if len(lines) > 20 {
        lines = lines[len(lines)-20:]
    }
    return &CommandResult{Output: fmt.Sprintf("[state=%s]\n%s",
        state, strings.Join(lines, "\n"))}, nil
}

func (c *TasksCommand) stop(id string) (*CommandResult, error) {
    if err := c.manager.StopTask(id); err != nil {
        return nil, err
    }
    return &CommandResult{Output: fmt.Sprintf("stopped %s",
        id)}, nil
}

```



Step 5: Run slash tests

```
cd HelixCode && go test -count=1 -race -run "TestSlashTasks" ./
    internal/commands/
```

Expected: PASS (5/5).



Step 6: Add RegisterBuiltinCommandsWithTasks

Modify `helix_code/internal/commands/builtin/register.go` to add (mirroring the existing `RegisterBuiltinCommandsWithMCP` pattern from F06):

```
// Add the workflow + commands imports if missing.
import (
    "dev.helix.code/internal/commands"
    "dev.helix.code/internal/workflow"
)

// RegisterBuiltinCommandsWithTasks registers all built-in
// commands plus the
// /tasks command bound to the given BackgroundManager.
func RegisterBuiltinCommandsWithTasks(registry
    *commands.Registry, mgr *workflow.BackgroundManager)
    error {
    if err := RegisterBuiltinCommands(registry); err != nil {
        return err
    }
    return registry.Register(commands.NewTasksCommand(mgr))
}
```

Then update `GetBuiltinCommandNames()` to include "tasks". If the existing `TestRegisterBuiltinCommands` iterates over names and expects each to be registered without args, add "tasks" to its skip set the same way "mcp"/"hooks"/"worktree" are skipped.



Step 7: Add register test

Create `helix_code/internal/commands/builtin/tasks_register_test.go`:

```
package builtin

import (
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    "go.uber.org/zap"

    "dev.helix.code/internal/commands"
    "dev.helix.code/internal/workflow"
)

func TestRegisterBuiltinCommandsWithTasks(t *testing.T) {
    registry := commands.NewRegistry()
```

```

    bm := workflow.NewBackgroundManager(zap.NewNop(),
        workflow.ManagerConfig{})
    defer bm.Close()
    require.NoError(t,
        RegisterBuiltinCommandsWithTasks(registry, bm))

    cmd, ok := registry.Get("tasks")
    require.True(t, ok)
    assert.Equal(t, "tasks", cmd.Name())
}

```

If `commands.Registry`'s lookup uses `Lookup` instead of `Get` (or a different method), adapt accordingly.



Step 8: Run all command tests

```

cd HelixCode && go test -count=1 -race ./internal/commands/ ./
internal/commands/builtin/

```

Expected: PASS — including `TestSlashTasks_*`, `TestRegisterBuiltinCommandsWithTasks`, and existing `Slash/Register` tests for `hooks/worktree/mcp`.



Step 9: Anti-bluff smoke

```

cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" internal/commands/tasks_command.go internal/
commands/builtin/register.go && echo "BLUFF FOUND" ||
echo "clean"

```

Expected: clean.



Step 10: Commit

```

git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    helix_code/internal/commands/tasks_command.go helix_code/
    internal/commands/tasks_command_test.go helix_code/
    internal/commands/builtin/register.go helix_code/
    internal/commands/builtin/tasks_register_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F07-T08): add /tasks slash command + builtin registration
helper

```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

```

EOF
)"

```

Task 9: cmd/cli/main.go startup wiring + integration test (real subprocess)

Files: - Modify: `helix_code/cmd/cli/main.go` — construct `BackgroundManager`, wire into `ToolRegistry + RegisterBuiltinCommandsWithTasks` - Create: `helix_code/tests/integration/background_shell_test.go`



Step 1: Wire BackgroundManager into main.go

Find the existing F06 wiring block in `helix_code/cmd/cli/main.go` where `mcp.NewManager()` is constructed and `RegisterBuiltinCommandsWithMCP` is called. Add adjacent:

```
// F07: background task manager
bgMgr := workflow.NewBackgroundManager(logger,
    workflow.ManagerConfig{})
defer bgMgr.Close()
toolRegistry.SetBackgroundManager(bgMgr)
toolRegistry.RegisterTaskTools(bgMgr)
if err := builtin.RegisterBuiltinCommandsWithTasks(cmdRegistry,
    bgMgr); err != nil {
    log.Printf("tasks: register slash command failed: %v", err)
}
```

The exact variable names (`logger`, `toolRegistry`, `cmdRegistry`) must match what's already in `main.go` — read it first. Use `dev.helix.code/internal/workflow` and `dev.helix.code/internal/commands/builtin` imports if not already present.



Step 2: Verify cmd/cli/main.go still builds

```
cd HelixCode && go build ./cmd/cli/...
```

Expected: success.



Step 3: Write integration test

Create `helix_code/tests/integration/background_shell_test.go`:

```
//go:build integration

package integration

import (
    "context"
    "os/exec"
    "runtime"
    "strings"
    "testing"
```

```

    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    "go.uber.org/zap"

    "dev.helix.code/internal/tools"
    "dev.helix.code/internal/workflow"
)

func skipIfWindows(t *testing.T) {
    t.Helper()
    if runtime.GOOS == "windows" {
        t.Skip("SKIP-OK: shell-based integration tests are POSIX-
            only on this branch")
    }
}

// TestBackground_ShellEcho_StreamsAndCompletes goes through the
// real
// ToolRegistry + ToolRegistry.Execute path with
// run_in_background:true,
// using the real shell BackgroundAware adapter against a real
// subprocess.
func TestBackground_ShellEcho_StreamsAndCompletes(t *testing.T) {
    skipIfWindows(t)
    reg, err :=
        tools.NewToolRegistry(tools.DefaultRegistryConfig())
    require.NoError(t, err)
    bm := workflow.NewBackgroundManager(zap.NewNop(),
        workflow.ManagerConfig{})
    defer bm.Close()
    reg.SetBackgroundManager(bm)

    ctx, cancel := context.WithTimeout(context.Background(),
        10*time.Second)
    defer cancel()
    res, err := reg.Execute(ctx, "Bash", map[string]interface{}{
        "command": "echo hello",
        "run_in_background": true,
    })
    require.NoError(t, err)
    m := res.(map[string]interface{})
    taskID := m["task_id"].(string)
    require.Eventually(t, func() bool {
        task, err := bm.GetTask(taskID)
        return err == nil && task.State() ==
            workflow.TaskCompleted
    }, 10*time.Second, 100*time.Millisecond)
}

```

```

    }, 5*time.Second, 25*time.Millisecond)
    task, err := bm.GetTask(taskID)
    require.NoError(t, err)
    all := strings.Join(task.LastLines(100), "\n")
    assert.Contains(t, all, "hello")
}

// TestBackground_ShellSleep_StopCancels exercises the cancel
// path.
func TestBackground_ShellSleep_StopCancels(t *testing.T) {
    skipIfWindows(t)
    reg, err :=
        tools.NewToolRegistry(tools.DefaultRegistryConfig())
    require.NoError(t, err)
    bm := workflow.NewBackgroundManager(zap.NewNop(),
        workflow.ManagerConfig{})
    defer bm.Close()
    reg.SetBackgroundManager(bm)

    res, err := reg.Execute(context.Background(), "Bash",
        map[string]interface{}{
            "command": "sleep 30",
            "run_in_background": true,
        })
    require.NoError(t, err)
    m := res.(map[string]interface{})
    taskID := m["task_id"].(string)
    require.Eventually(t, func() bool {
        task, _ := bm.GetTask(taskID)
        return task != nil && task.State() ==
            workflow.TaskRunning
    }, 2*time.Second, 25*time.Millisecond)

    require.NoError(t, bm.StopTask(taskID))
    require.Eventually(t, func() bool {
        task, _ := bm.GetTask(taskID)
        return task != nil && (task.State() ==
            workflow.TaskCancelled || task.State() ==
            workflow.TaskFailed)
    }, 3*time.Second, 25*time.Millisecond)

    // Verify no orphan sleep process under our PID
    out, _ := exec.Command("pgrep", "-x", "sleep").Output()
    pids := strings.TrimSpace(string(out))
    if pids != "" {
        t.Logf("note: pgrep found sleep processes: %s (these may
            belong to other tests/users)", pids)
    }
}

```



```

}

// TestBackground_ConcurrentTasks proves multiple concurrent
// tasks all land.
func TestBackground_ConcurrentTasks(t *testing.T) {
    skipIfWindows(t)
    reg, err :=
        tools.NewToolRegistry(tools.DefaultRegistryConfig())
    require.NoError(t, err)
    bm := workflow.NewBackgroundManager(zap.NewNop(),
        workflow.ManagerConfig{})
    defer bm.Close()
    reg.SetBackgroundManager(bm)

    ids := make([]string, 0, 5)
    for i := 0; i < 5; i++ {
        res, err := reg.Execute(context.Background(), "Bash",
            map[string]interface{}{
                "command": "sleep 0.5",
                "run_in_background": true,
            })
        require.NoError(t, err)
        ids = append(ids, res.(map[string]interface{})
            ["task_id"].(string))
    }
    require.Eventually(t, func() bool {
        for _, id := range ids {
            task, err := bm.GetTask(id)
            if err != nil || task.State() !=
                workflow.TaskCompleted {
                return false
            }
        }
        return true
    }, 5*time.Second, 50*time.Millisecond)
    assert.Len(t, bm.ListTasks(), 5)
}

```



Step 4: Run integration test

```

cd HelixCode && go test -count=1 -tags=integration -run
    "TestBackground_" ./tests/integration/...

```

Expected: PASS (3/3).



Step 5: Full unit + integration sweep

```
cd HelixCode
go test -count=1 -race ./internal/workflow/... ./internal/
tools/ ./internal/tools/shell/ ./internal/commands/ ./
internal/commands/builtin/ ./cmd/cli/...
go test -count=1 -tags=integration -run "TestBackground_" ./
tests/integration/...
```

Pre-existing failures in tools/git, tools/multiedit, tools/shell tests unrelated to F07 may persist; document if so and do not introduce new failures in any package this task touches.



Step 6: Cross-compile check (linux native; Windows has pre-existing CGO failures)

```
cd HelixCode && go build ./cmd/cli/... ./internal/workflow/... ./
internal/tools/ ./internal/tools/shell/ ./internal/
commands/...
```

Expected: clean.



Step 7: Anti-bluff smoke (broadest scope for F07)

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" \
internal/workflow/ internal/tools/types_background.go internal/
tools/task_tools.go \
internal/tools/shell/background.go internal/commands/
tasks_command.go internal/commands/builtin/register.go \
cmd/cli/main.go && echo "BLUFF FOUND" || echo "clean"
```

Expected: clean.



Step 8: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
helix_code/cmd/cli/main.go helix_code/tests/integration/
background_shell_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
-m "$(cat <<'EOF'
feat(P1-F07-T09): wire BackgroundManager into cmd/cli startup +
integration test (real subprocess)
```

Co-Authorred-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

EOF

)"

Task 10: Challenge with runtime evidence + cross-compile check

Files: - Create: challenges/p1-f07-background-tasks/CHALLENGE.md (in the Challenges submodule) - Create: challenges/p1-f07-background-tasks/run.sh - Create: helix_code/tests/integration/cmd/p1f07_challenge/main.go — small harness that polls TaskOutput across a streaming task - Modify: docs/improvements/06_phase_1_evidence.md — append T10 section with pasted runtime output



Step 1: Write challenge harness

Create helix_code/tests/integration/cmd/p1f07_challenge/main.go:

```
// p1f07_challenge runs a real background task and prints its
// polling timeline,
// proving mid-execution streaming. It is the runtime-evidence
// harness for the
// F07 Challenge (Article XI §11.9).
package main

import (
    "context"
    "fmt"
    "os"
    "os/exec"
    "strings"
    "time"

    "go.uber.org/zap"

    "dev.helix.code/internal/tools"
    "dev.helix.code/internal/workflow"
)

func main() {
    if err := run(); err != nil {
        fmt.Fprintln(os.Stderr, "FAIL:", err)
        os.Exit(1)
    }
}

func run() error {
    reg, err :=
        tools.NewToolRegistry(tools.DefaultRegistryConfig())
    if err != nil {
        return fmt.Errorf("registry: %w", err)
    }
}
```

```

bm := workflow.NewBackgroundManager(zap.NewNop(),
    workflow.ManagerConfig{})
defer bm.Close()
reg.SetBackgroundManager(bm)

ctx := context.Background()

// Streaming task
fmt.Println("==> start background streaming task")
res, err := reg.Execute(ctx, "Bash", map[string]interface{}{
    "command":      "for i in 1 2 3; do echo line $i;
    sleep 0.3; done",
    "run_in_background": true,
})
if err != nil {
    return fmt.Errorf("execute: %w", err)
}
taskID := res.(map[string]interface{})["task_id"].(string)
fmt.Println("task_id =", taskID)

// Poll every 200ms for 3 seconds
deadline := time.Now().Add(3 * time.Second)
prevCount := -1
for time.Now().Before(deadline) {
    task, err := bm.GetTask(taskID)
    if err != nil {
        return fmt.Errorf("get task: %w", err)
    }
    lines := task.LastLines(100)
    if len(lines) != prevCount {
        fmt.Printf("[poll t=%dms] state=%s lines=%d -> %s\n",
            time.Since(task.StartedAt).Milliseconds(),
            task.State(), len(lines), strings.Join(lines,
                " | "))
        prevCount = len(lines)
    }
    if task.State() == workflow.TaskCompleted {
        break
    }
    time.Sleep(200 * time.Millisecond)
}
task, _ := bm.GetTask(taskID)
if task.State() != workflow.TaskCompleted {
    return fmt.Errorf("task did not complete in 3s,
        state=%s", task.State())
}
if prevCount < 2 {

```

```

        return fmt.Errorf("only saw %d lines; streaming did not
        work as expected", prevCount)
    }
    fmt.Println("==> streaming verified: agent saw growing line
        count mid-execution")

    // Cancel task
    fmt.Println("==> start sleep 30 task and cancel")
    res2, err := reg.Execute(ctx, "Bash", map[string]interface{}{
        "command": "sleep 30",
        "run_in_background": true,
    })
    if err != nil {
        return fmt.Errorf("execute sleep: %w", err)
    }
    id2 := res2.(map[string]interface{})["task_id"].(string)
    time.Sleep(200 * time.Millisecond)
    if err := bm.StopTask(id2); err != nil {
        return fmt.Errorf("stop task: %w", err)
    }
    cancelDeadline := time.Now().Add(3 * time.Second)
    for time.Now().Before(cancelDeadline) {
        task, _ := bm.GetTask(id2)
        if task != nil && (task.State() ==
            workflow.TaskCancelled || task.State() ==
            workflow.TaskFailed) {
            fmt.Println("==> sleep task cancelled, state=",
                task.State())
            break
        }
        time.Sleep(50 * time.Millisecond)
    }
    task2, _ := bm.GetTask(id2)
    if task2 == nil || task2.State() != workflow.TaskCancelled
        && task2.State() != workflow.TaskFailed {
        return fmt.Errorf("sleep task not cancelled within 3s")
    }

    // Check no stray sleep PID
    out, _ := exec.Command("pgrep", "-x", "sleep").Output()
    pids := strings.TrimSpace(string(out))
    fmt.Println("==> pgrep -x sleep returned:", pids)
    // Note: not failing on stray PIDs because other tests/users
    // may have unrelated sleeps;
    // the cancellation evidence is the state transition above.

    fmt.Println("==> P1-F07 challenge harness PASS")

```

```
    return nil
}
```



Step 2: Write CHALLENGE.md

Create challenges/p1-f07-background-tasks/CHALLENGE.md:

Challenge: P1-F07 — Background Task System

Purpose

Prove that HelixCode's background task system actually streams mid-execution output and successfully cancels long-running shell processes. Per Article XI §11.9, every PASS must carry positive runtime evidence.

Procedure

1. Build the F07 challenge harness (``tests/integration/cmd/p1f07_challenge``).
2. Run the harness — it:
 - a. Starts a Bash command that emits 3 lines with 0.3s gaps.
 - b. Polls TaskOutput every 200ms; logs each new state/line count to stdout.
 - c. Asserts the polling timeline shows growing line counts (not just final).
 - d. Starts ``sleep 30``, cancels it after 200ms, asserts cancel within 3s.
 - e. Logs ``pgrep -x sleep`` output as supporting evidence.
3. Anti-bluff smoke: ``grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" helix_code/internal/workflow/helix_code/internal/tools/task_tools.go helix_code/internal/tools/shell/background.go helix_code/internal/commands/tasks_command.go`` returns empty.
4. Cross-compile linux: ``cd HelixCode && go build ./cmd/cli/... ./internal/workflow/... ./internal/tools/...``.

Pass criteria

- Harness exits 0 with ``==> P1-F07 challenge harness PASS`` as final line.
- Polling timeline shows ≥ 2 distinct line counts during execution (proves streaming).
- Sleep task transitions to Cancelled or Failed within 3s of StopTask.

- Anti-bluff smoke clean.
- Cross-compile linux clean.



Step 3: Write run.sh

Create challenges/p1-f07-background-tasks/run.sh:

```
#!/usr/bin/env bash
set -euo pipefail
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
ROOT="$(cd "$HERE/../../" && pwd)"
cd "$ROOT/HelixCode"

echo "==> build F07 challenge harness"
HARNESS_BIN="$(mktemp -d)/p1f07_challenge"
go build -o "$HARNESS_BIN" ./tests/integration/cmd/
p1f07_challenge

echo "==> run harness"
"$HARNESS_BIN"

echo "==> anti-bluff smoke on F07-affected code"
if grep -rn "simulated\|for now\|TODO implement\|placeholder" \
    internal/workflow/ \
    internal/tools/types_background.go \
    internal/tools/task_tools.go \
    internal/tools/shell/background.go \
    internal/commands/tasks_command.go; then
    echo "BLUFF FOUND" >&2
    exit 1
fi
echo "clean"

echo "==> cross-compile linux"
go build ./cmd/cli/... ./internal/workflow/... ./internal/tools/

echo "==> P1-F07 challenge PASS"
```



Step 4: chmod and run

```
chmod +x /run/media/milosvasic/DATA4TB/Projects/helix_code/
challenges/p1-f07-background-tasks/run.sh
/run/media/milosvasic/DATA4TB/Projects/helix_code/challenges/p1-
f07-background-tasks/run.sh 2>&1 | tee /tmp/p1f07-run.log
```

Expected: exits 0 with final line ==> P1-F07 challenge PASS.



Step 5: Append runtime evidence

In docs/improvements/06_phase_1_evidence.md, under the F07 section header (added in T01), append:

T10 – Challenge run

```
````bash
$./challenges/p1-f07-background-tasks/run.sh
[paste FULL contents of /tmp/p1f07-run.log here verbatim – every
 line]
````
```

T10 – All commits in the F07 branch

```
````bash
$ git log --oneline | grep "P1-F07"
[paste actual output of: cd /run/media/milosvasic/DATA4TB/
 Projects/HelixCode && git log --oneline | grep "P1-F07"]
````
```

Use the actual content from /tmp/p1f07-run.log and the actual git log output. NO PARAPHRASING. NO FABRICATION.



Step 6: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
    challenges/p1-f07-background-tasks/ helix_code/tests/
    integration/cmd/p1f07_challenge docs/improvements/
    06_phase_1_evidence.md
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
    -m "$(cat <<'EOF'
feat(P1-F07-T10): challenge with runtime evidence + cross-compile
    check
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

```
EOF
)"
```

If challenges/ is a submodule (it is, per the F06 close-out experience), the commit may need to be made inside the submodule first, then the meta-repo commits the updated submodule pointer + evidence.md. Match the F06-T13 dual-commit pattern: submodule commit for challenges/p1-f07-background-tasks/ files, meta-repo commit for the submodule pointer + docs/improvements/06_phase_1_evidence.md.

Task 11: Feature 7 close-out + push to 4 remotes

Files: - Modify: docs/improvements/PROGRESS.md



Step 1: Update PROGRESS.md current focus

Replace the F07 active block with:

Current focus

- **Active phase:** P1 – claude-code feature porting
- **Active feature:** (idle, awaiting next feature pick – F08 candidate)
- **Active task:** –
- **Last completed:** P1-F07-T11 – Feature 7 (Background Task System) close-out + push
- **Owner:** agent (Claude Opus 4.7)
- **Started:** 2026-05-04
- **Last touched:** 2026-05-05
- **Blocked-on:** none



Step 2: Tick all F07 task list items

In the existing P1-F07 task list block, change every - [] to - [x] for items T01–T11.



Step 3: Final verification sweep

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
```

Unit + race

```
go test -count=1 -race ./internal/workflow/... ./internal/
tools/ ./internal/tools/shell/ ./internal/commands/ ./
internal/commands/builtin/ ./cmd/cli/...
```

Integration

```
go test -count=1 -tags=integration -run "TestBackground_" ./
tests/integration/...
```

Anti-bluff

```
grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" \
internal/workflow/ internal/tools/types_background.go internal/
tools/task_tools.go \
internal/tools/shell/background.go internal/commands/
tasks_command.go internal/commands/builtin/register.go \
cmd/cli/main.go && echo "BLUFF FOUND" || echo "clean"
```

Cross-compile linux

```
go build ./cmd/cli/... ./internal/workflow/... ./internal/tools/
```

```
# go vet
go vet ./internal/workflow/... ./internal/tools/ ./internal/
      tools/shell/ ./internal/commands/... ./cmd/cli/...

# Final challenge re-run
chmod +x /run/media/milosvasic/DATA4TB/Projects/helix_code/
      challenges/p1-f07-background-tasks/run.sh
/run/media/milosvasic/DATA4TB/Projects/helix_code/challenges/p1-
      f07-background-tasks/run.sh
```

If any step fails, STOP and report. Do NOT push.



Step 4: Commit close-out

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add docs/
      improvements/PROGRESS.md
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
      -m "$(cat <<'EOF'
chore(P1-F07-T11): Feature 7 (Background Task System) close-out

Co-Authored-By: Claude Opus 4.7 (1M context)
      <noreply@anthropic.com>
EOF
)"
```



Step 5: Push non-force to all 4 remotes (programme convention)

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
      origin main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
      github main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
      gitlab main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
      upstream main
```

CRITICAL CONSTRAINTS for the push: - NO --force. NO --force-with-lease. NO history rewrites. (CONST-043) - If a push is rejected non-fast-forward, STOP and report — DO NOT force. - Each push must produce real output (commit hash → branch advance), not an error.

Self-review notes

1. **Spec coverage:** every section of the spec has at least one task — types/state machine (T02), manager + sweeper (T03), BackgroundAware interface (T04), shell adapter (T05), registry dispatch (T06), agent tools (T07), /tasks slash + builtin reg (T08), main.go wiring + integration (T09), Challenge + cross-compile (T10), close-out + push (T11).

2. **TDD discipline:** every code-introducing task starts with a failing test (Step 1), runs to confirm failure (Step 2), implements minimally (Step 3+), reruns to confirm pass.
3. **Type consistency:** `BackgroundManager`, `BackgroundTask`, `LineSink`, `BackgroundExecutor`, `ManagerConfig`, `TaskState` (and constants `Task*`), `BackgroundAware`, `ErrNoBackgroundMgr`, `ErrTaskNotFound`, `ErrTaskNotRunning`, `ErrManagerClosed`, `ErrTooManyTasks`, `TaskOutputTool`, `TaskStopTool`, `TasksCommand`, `RegisterBuiltinCommandsWithTasks` are spelled identically across all tasks. The two `LineSink` types (one in `internal/tools/`, one in `internal/workflow/`) have the same shape; the registry adapter explicitly converts between them.
4. **Cross-platform:** all new code uses `os/exec + bufio.Scanner + ctx-driven exec.CommandContext`; no platform-specific imports beyond what the existing shell package already uses. Cross-compile check appears in T05, T09, T10, T11.
5. **Anti-bluff:** every task that introduces code runs the FULL 4-term smoke pattern. The Challenge in T10 captures real runtime evidence with multi-poll streaming visibility per Article XI §11.9.
6. **No new external dependencies:** `uuid + zap + cobra + testify` all already in `go.mod`.
7. **Branch + push:** stays on `main`, pushes non-force to all four remotes per the programme convention validated through F01–F06.
8. **Pre-existing failures:** `internal/tools/git`, `internal/tools/multiedit`, `internal/tools/shell` tests have flakiness/issues from before F07 — documented as out-of-scope. New tasks must not introduce additional failures in any package they touch.